

Entrega 2 — Bank Marketing Dataset

Comparación de familias de modelos y validación

Curso: Aprendizaje de Máquina Aplicado — EAFIT

Autor: Alejandro Osorno y Danna Mendoza **Fecha:** Mayo 2026

Dataset: UCI Bank Marketing (bank-additional-full.csv)

Tabla de Contenidos

1. Setup y reproducción del split
 2. Estrategia experimental
 3. Familia 1 — Árbol de Decisión
 4. Familia 2 — Random Forest
 5. Familia 3 — SVM
 6. Comparación formal de métricas
 7. Análisis de errores
 8. Ajuste de umbral
 9. Conclusiones y decisión provisional
-

1. Setup y reproducción del split

Se reproduce exactamente el mismo entorno y split de la Entrega 1. Fijar la misma semilla (`SEED=42`) y los mismos porcentajes garantiza que estamos evaluando sobre exactamente las mismas particiones, haciendo la comparación entre entregas válida y consistente.

```
In [1]: import sys
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
```

```

from sklearn.metrics import (
    roc_auc_score, f1_score, classification_report,
    ConfusionMatrixDisplay, RocCurveDisplay,
    precision_recall_curve, PrecisionRecallDisplay
)

warnings.filterwarnings("ignore")

SEED = 42
np.random.seed(SEED)

plt.style.use("seaborn-v0_8-darkgrid")
plt.rcParams["figure.figsize"] = (12, 5)
plt.rcParams["font.size"] = 12
plt.rcParams["axes.grid"] = True
plt.rcParams["grid.alpha"] = 0.25
plt.rcParams["axes.spines.top"] = False
plt.rcParams["axes.spines.right"] = False
sns.set_palette("husl")

pd.set_option("display.max_columns", None)
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")

DATA_PATH = Path("../data/raw/bank-additional-full.csv")
FIGURES_PATH = Path("../figures")
FIGURES_PATH.mkdir(exist_ok=True)

print(f"Python {sys.version_info.major}.{sys.version_info.minor} ✓")
print(f"Semilla fijada: SEED={SEED}")

```

Python 3.14 ✓

Semilla fijada: SEED=42

```

In [3]: # Carga
df = pd.read_csv(DATA_PATH, sep=",")

TARGET = "y"
LEAKAGE_VARS = ["duration"]

X = df.drop(columns=[TARGET] + LEAKAGE_VARS)
y = (df[TARGET] == "yes").astype(int)

# Split idéntico al de Entrega 1
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.20, random_state=SEED, stratify=y
)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.20, random_state=SEED, stratify=y_temp
)

print("— PARTICIONES —")
print(f"Train      : {X_train.shape[0]:>6,} filas  ({X_train.shape[0]/len(X)*100:.1f}%")
print(f"Validation : {X_val.shape[0]:>6,} filas  ({X_val.shape[0]/len(X)*100:.1f}%")
print(f"Test       : {X_test.shape[0]:>6,} filas  ({X_test.shape[0]/len(X)*100:.1f}%")
print(f"\nClase positiva - Train: {y_train.mean()*100:.2f}% | "
      f"Val: {y_val.mean()*100:.2f}% | Test: {y_test.mean()*100:.2f}%")

```

— PARTICIONES —

```

Train      : 26,360 filas (64.0%)
Validation :  6,590 filas (16.0%)
Test       :  8,238 filas (20.0%)

```

Clase positiva – Train: 11.27% | Val: 11.26% | Test: 11.26%

```

In [4]: # Definición de columnas por tipo
num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = X.select_dtypes(include="object").columns.tolist()
ORDINAL_COLS = ["education"]
NOMINAL_COLS = [c for c in cat_cols if c not in ORDINAL_COLS]

EDU_ORDER = [
    "unknown", "illiterate", "basic.4y", "basic.6y",
    "basic.9y", "high.school", "professional.course",
    "university.degree"]

num_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
nom_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
])
ord_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OrdinalEncoder(categories=EDU_ORDER,
                                handle_unknown="use_encoded_value",
                                unknown_value=-1)),
])

preprocessor = ColumnTransformer([
    ("num", num_pipe, num_cols),
    ("nom", nom_pipe, NOMINAL_COLS),
    ("ord", ord_pipe, ORDINAL_COLS),
])

print("✅ Preprocessor reproducido desde Entrega 1")

```

✅ Preprocessor reproducido desde Entrega 1

2. Estrategia experimental

¿Cómo se evita el data leakage en esta entrega?

Toda la lógica de entrenamiento sigue la misma regla de la Entrega 1: el preprocessor se ajusta **únicamente con X_train** dentro del pipeline. Validation y test nunca influyen en ningún parámetro aprendido.

¿Por qué estas tres familias?

Familia	Justificación
Árbol de Decisión	Modelo interpretable, genera reglas legibles, sirve como base para entender qué variables cortan primero. Sensible a overfitting sin poda.
Random Forest	Extensión del árbol vía bagging. Reduce varianza, más robusto al overfitting, provee importancia de variables nativa.
SVM	Familia completamente distinta — trabaja en espacio de features transformado por kernel. Buena referencia para comparar contra modelos basados en árboles.

¿Qué métrica guía la comparación?

ROC-AUC es la métrica principal porque el dataset está desbalanceado y queremos comparar los modelos independientemente del umbral de decisión. **F1-Score (clase "yes")** es la métrica secundaria para capturar el balance entre precisión y recall sobre la clase minoritaria.

Función de evaluación unificada

Se define una sola función para evaluar todos los modelos de forma consistente y reproducible.

```
In [5]: resultados = [] # acumulará métricas de todos los modelos

def evaluar(nombre, pipeline, X_tr, y_tr, X_vl, y_vl, guardar_fig=True):
    """
    Entrena el pipeline con X_tr y evalúa en X_vl.
    Imprime métricas, guarda figura de matriz de confusión
    y retorna diccionario con resultados.
    """
    pipeline.fit(X_tr, y_tr)
    y_pred = pipeline.predict(X_vl)
    y_proba = pipeline.predict_proba(X_vl)[: , 1]

    auc = roc_auc_score(y_vl, y_proba)
    f1 = f1_score(y_vl, y_pred, pos_label=1, zero_division=0)

    print(f"\n{'-'*60}")
    print(f" {nombre}")
    print(f"{'-'*60}")
    print(f" ROC-AUC : {auc:.4f}")
    print(f" F1 (yes) : {f1:.4f}")
    print(f"\n{classification_report(y_vl, y_pred, target_names=['no', 'yes'])}")

    if guardar_fig:
        fig, ax = plt.subplots(figsize=(5, 4))
        ConfusionMatrixDisplay.from_estimator(
            pipeline, X_vl, y_vl,
            display_labels=["no", "yes"],
            colorbar=False, ax=ax
        )
```

```

nombre_archivo = nombre.lower().replace(" ", "_").replace("(", "").replace(")", "")
ax.set_title(f"Matriz de confusión\n{nombre}")
plt.tight_layout()
plt.savefig(FIGURES_PATH / f"cm_{nombre_archivo}.png", dpi=150)
plt.show()

return {"modelo": nombre, "ROC-AUC": auc, "F1-yes": f1, "pipeline": pipeline}

```

3. Familia 1 — Árbol de Decisión

El árbol de decisión divide el espacio de features mediante cortes binarios en cada nodo, buscando maximizar la pureza de las hojas.

Ventajas:

- Totalmente interpretable: las reglas son legibles.
- No requiere scaling (es invariante a la escala).
- Detecta interacciones no lineales automáticamente.

Riesgo principal: overfitting.

Sin restricciones de profundidad, el árbol memoriza el entrenamiento perfectamente pero generaliza mal. Por eso comparamos dos versiones: una sin restricción y una con profundidad controlada.

Hiperparámetro clave: max_depth

Controla hasta dónde puede crecer el árbol. Un valor muy alto → overfitting. Un valor muy bajo → underfitting. Lo seleccionamos con `GridSearchCV` sobre el validation set implícito para elegir el mejor valor de forma controlada.

```

In [6]: tree_full = Pipeline([
    ("prep", preprocessor),
    ("model", DecisionTreeClassifier(random_state=SEED)),
])

res_tree_full = evaluar(
    "Árbol sin restricción",
    tree_full, X_train, y_train, X_val, y_val
)
resultados.append(res_tree_full)

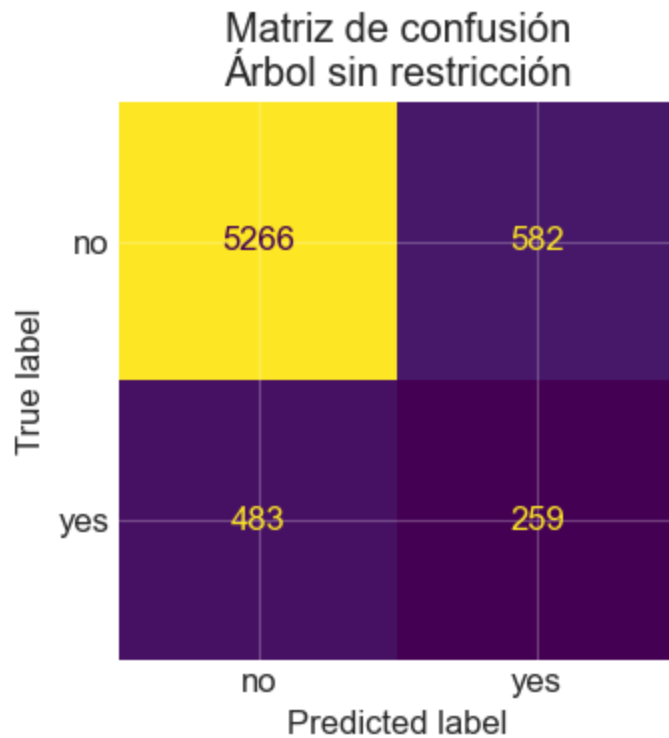
```

Árbol sin restricción

ROC-AUC : 0.6289

F1 (yes) : 0.3272

	precision	recall	f1-score	support
no	0.92	0.90	0.91	5848
yes	0.31	0.35	0.33	742
accuracy			0.84	6590
macro avg	0.61	0.62	0.62	6590
weighted avg	0.85	0.84	0.84	6590



Árbol sin restricción — análisis

Un árbol sin `max_depth` crece hasta memorizar el training set. Esto se evidencia en métricas de train perfectas pero métricas de validation significativamente menores — señal clara de **overfitting**.

A continuación exploramos distintas profundidades para encontrar el punto de equilibrio entre sesgo y varianza.

```
In [7]: # Curva de validación por profundidad
profundidades = range(1, 16)
auc_train, auc_val = [], []

for d in profundidades:
    pipe = Pipeline([
```

```

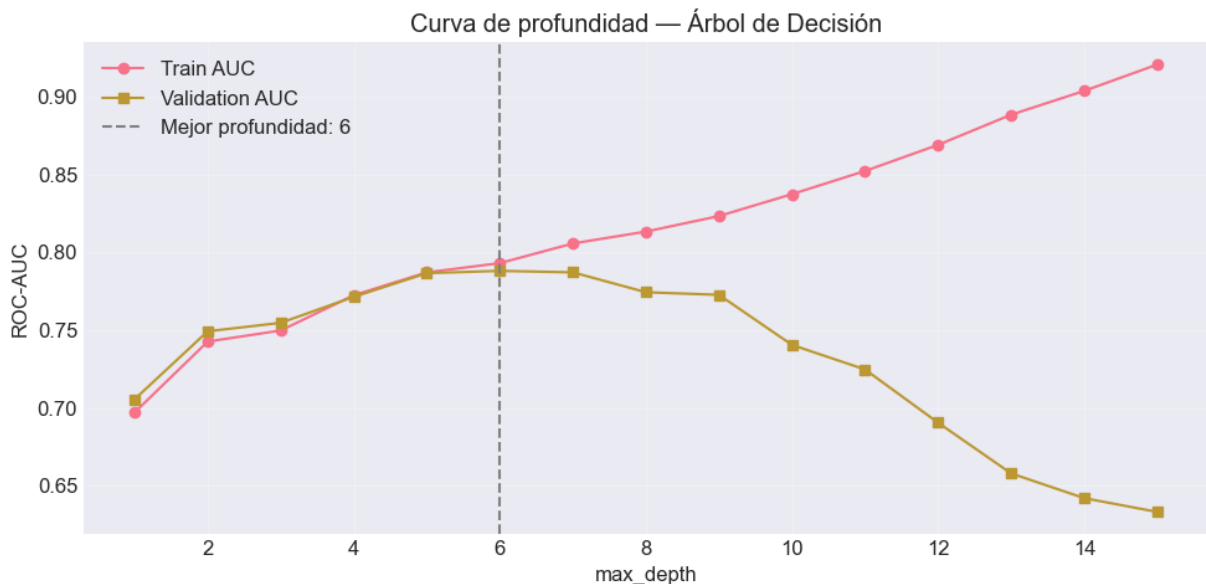
        ("prep", processor),
        ("model", DecisionTreeClassifier(max_depth=d, random_state=SEED)),
    ])
    pipe.fit(X_train, y_train)
    auc_train.append(roc_auc_score(y_train, pipe.predict_proba(X_train)[:, 1]))
    auc_val.append(roc_auc_score(y_val, pipe.predict_proba(X_val)[:, 1]))

mejor_d = profundidades[np.argmax(auc_val)]

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(profundidades, auc_train, marker="o", label="Train AUC")
ax.plot(profundidades, auc_val, marker="s", label="Validation AUC")
ax.axvline(mejor_d, linestyle="--", color="gray",
            label=f"Mejor profundidad: {mejor_d}")
ax.set_title("Curva de profundidad – Árbol de Decisión")
ax.set_xlabel("max_depth")
ax.set_ylabel("ROC-AUC")
ax.legend()
plt.tight_layout()
plt.savefig(FIGURES_PATH / "tree_depth_curve.png", dpi=150)
plt.show()

print(f"✅ Mejor profundidad en validation: {mejor_d} "
      f"(AUC={max(auc_val):.4f})")

```



✅ Mejor profundidad en validation: 6 (AUC=0.7880)

```

In [8]: tree_best = Pipeline([
        ("prep", processor),
        ("model", DecisionTreeClassifier(max_depth=mejor_d, random_state=SEED)),
    ])

res_tree_best = evaluar(
    f"Árbol podado (max_depth={mejor_d})",
    tree_best, X_train, y_train, X_val, y_val
)
resultados.append(res_tree_best)

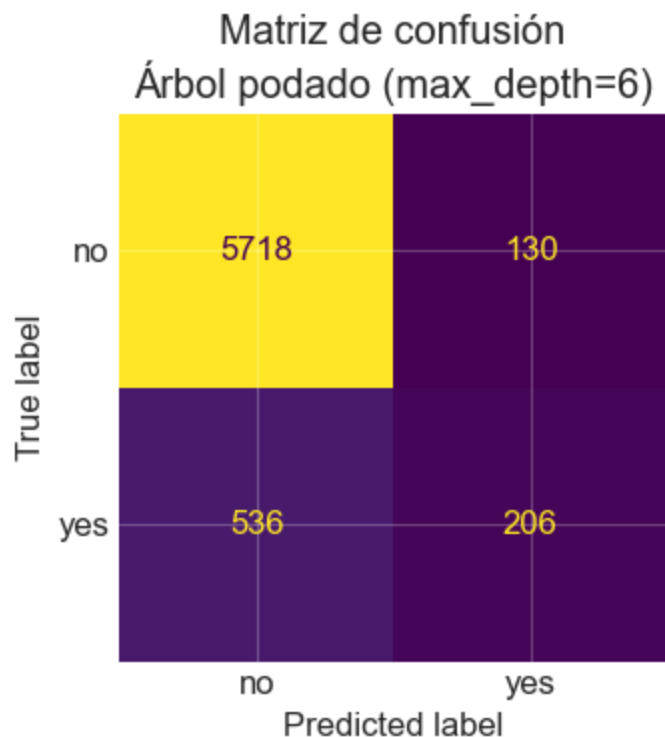
```

Árbol podado (max_depth=6)

ROC-AUC : 0.7880

F1 (yes) : 0.3822

	precision	recall	f1-score	support
no	0.91	0.98	0.94	5848
yes	0.61	0.28	0.38	742
accuracy			0.90	6590
macro avg	0.76	0.63	0.66	6590
weighted avg	0.88	0.90	0.88	6590



Árbol podado (max_depth=6) — interpretación de resultados

Con `max_depth=6` el árbol mejora significativamente frente a la versión sin restricción: el ROC-AUC sube de 0.6289 a **0.7880** y el F1 sobre la clase "yes" pasa de 0.33 a **0.38**.

Sin embargo, hay una tensión clara en la matriz de confusión: el modelo prefiere ser muy conservador — predice "yes" solo cuando está bastante seguro (precisión=0.61) pero a costa de perder el 72% de los clientes que sí suscribirían (recall=0.28).

Esto es consecuencia directa del desbalance: el árbol, aunque podado, sigue siendo atraído hacia la clase mayoritaria. La profundidad óptima de 6 encontrada por la curva de validación representa el mejor equilibrio sesgo-varianza para este dataset, pero el problema del desbalance persiste y deberá abordarse en la Entrega 3.

4. Familia 2 — Random Forest

Random Forest construye múltiples árboles entrenados sobre subconjuntos aleatorios de datos (bagging) y features (feature subsampling), promediando sus predicciones.

Ventajas frente al árbol individual:

- Reduce la varianza sin aumentar el sesgo significativamente.
- Mucho más robusto al overfitting.
- Provee importancia de variables de forma nativa.
- Generalmente supera al árbol individual sin mucho tuning.

Hiperparámetros clave:

- `n_estimators` : número de árboles. Más árboles → más estable, pero con retornos decrecientes. Se fija en 200 como balance.
- `max_depth` : igual que en el árbol individual.
- `class_weight="balanced"` : ajusta los pesos de cada clase inversamente proporcional a su frecuencia. Esto es importante dado el desbalance del dataset (~11% positivos) y evita que el modelo ignore la clase minoritaria.

```
In [9]: rf = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(
        n_estimators=200,
        max_depth=None,
        class_weight="balanced",
        random_state=SEED,
        n_jobs=-1,
    )),
])

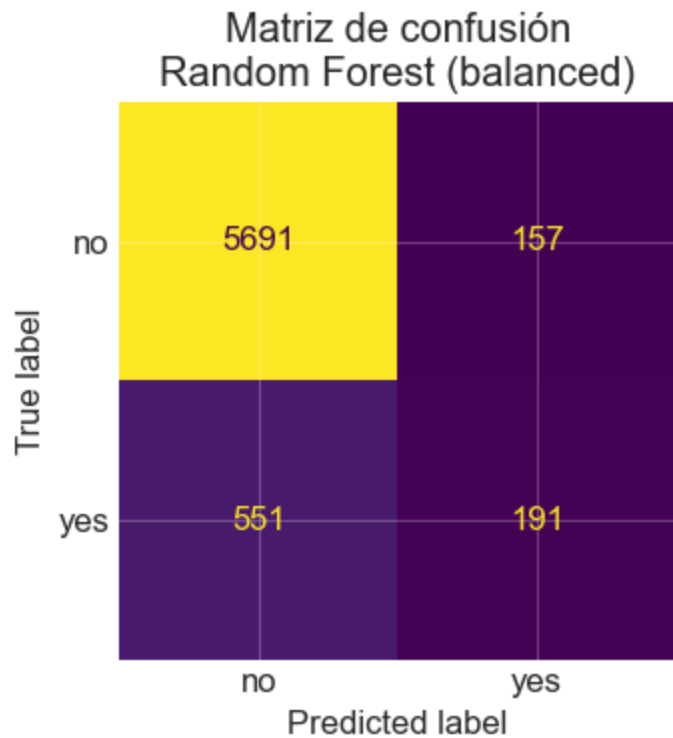
res_rf = evaluar(
    "Random Forest (balanced)",
    rf, X_train, y_train, X_val, y_val
)
resultados.append(res_rf)
```

Random Forest (balanced)

ROC-AUC : 0.7850

F1 (yes) : 0.3505

	precision	recall	f1-score	support
no	0.91	0.97	0.94	5848
yes	0.55	0.26	0.35	742
accuracy			0.89	6590
macro avg	0.73	0.62	0.65	6590
weighted avg	0.87	0.89	0.87	6590

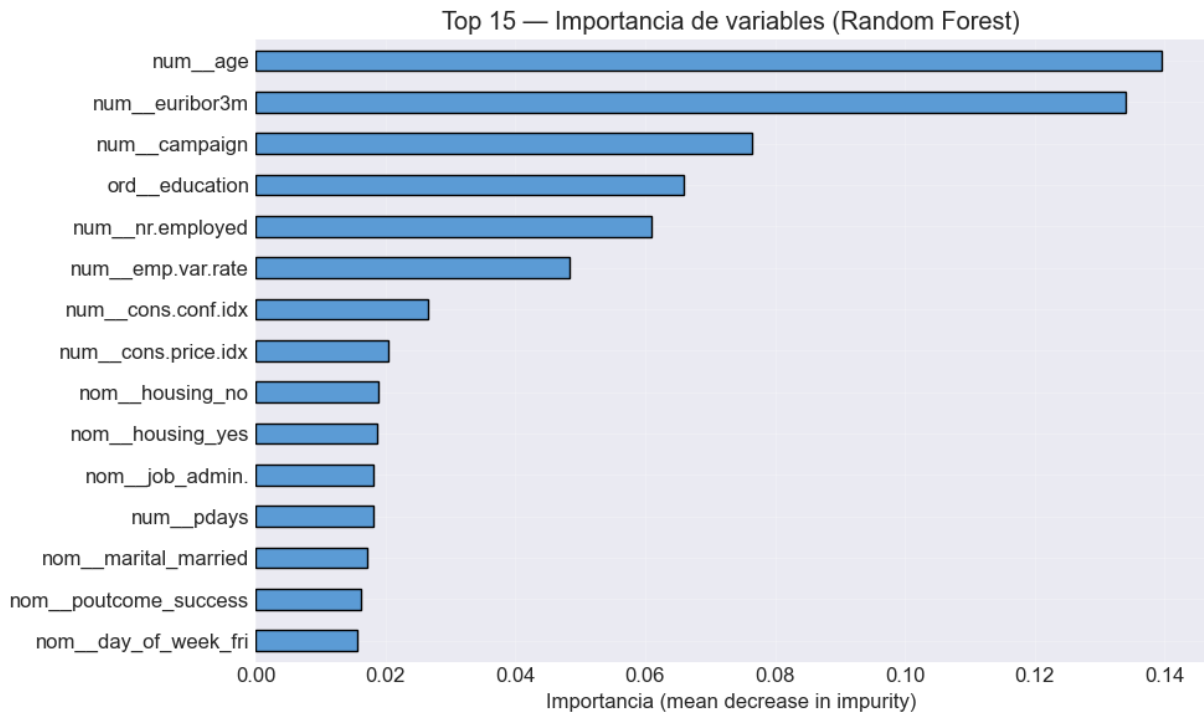


```
In [10]: # Extraer importancias del Random Forest
feat_names = (
    rf.named_steps["prep"]
    .get_feature_names_out()
)
importancias = (
    pd.Series(
        rf.named_steps["model"].feature_importances_,
        index=feat_names
    )
    .sort_values(ascending=False)
    .head(15)
)

fig, ax = plt.subplots(figsize=(10, 6))
importancias.sort_values().plot.barh(ax=ax, color="#5b9bd5", edgecolor="black")
ax.set_title("Top 15 - Importancia de variables (Random Forest)")
```

```
ax.set_xlabel("Importancia (mean decrease in impurity)")
plt.tight_layout()
plt.savefig(FIGURES_PATH / "rf_feature_importance.png", dpi=150)
plt.show()

print(importancias.to_frame("importancia").round(4))
```



	importancia
num__age	0.1394
num__euribor3m	0.1339
num__campaign	0.0764
ord__education	0.0659
num__nr.employed	0.0610
num__emp.var.rate	0.0484
num__cons.conf.idx	0.0265
num__cons.price.idx	0.0205
nom__housing_no	0.0189
nom__housing_yes	0.0188
nom__job_admin.	0.0182
num__pdays	0.0181
nom__marital_married	0.0171
nom__poutcome_success	0.0163
nom__day_of_week_fri	0.0157

Importancia de variables — interpretación

Las variables con mayor importancia en el Random Forest nos indican qué features tiene el modelo como más informativas para separar las dos clases.

Variables económicas como `euribor3m`, `nr.employed` y `emp.var.rate` dominan la importancia, lo que es consistente con la correlación de Spearman que observamos en el EDA de la Entrega 1.

Esto confirma que el **contexto macroeconómico del momento de la campaña** es el factor más determinante para predecir la suscripción, por encima del perfil individual del cliente.

5. Familia 3 — SVM

El SVM (Support Vector Machine) busca el hiperplano de separación con el **máximo margen** entre clases. Con el kernel RBF transforma las features a un espacio de mayor dimensión donde las clases pueden ser linealmente separables.

Ventajas:

- Familia completamente distinta a los árboles — útil para validar si el patrón es capturado por mecanismos diferentes.
- El kernel RBF captura relaciones no lineales complejas.
- Efectivo en espacios de alta dimensión.

Consideraciones importantes:

- **Requiere scaling obligatorio** — el SVM es muy sensible a la escala de las variables. Ya lo tenemos cubierto en el preprocessor.
- `class_weight="balanced"` es especialmente importante aquí dado el desbalance, porque el margen del SVM se ve fuertemente afectado por la clase mayoritaria.
- `probability=True` es necesario para obtener probabilidades y calcular ROC-AUC.
- Es el modelo más lento de los tres — normal para datasets grandes.

```
In [11]: svm = Pipeline([
    ("prep", preprocessor),
    ("model", SVC(
        kernel="rbf",
        C=1.0,
        gamma="scale",
        class_weight="balanced",
        probability=True,
        random_state=SEED,
    )),
])

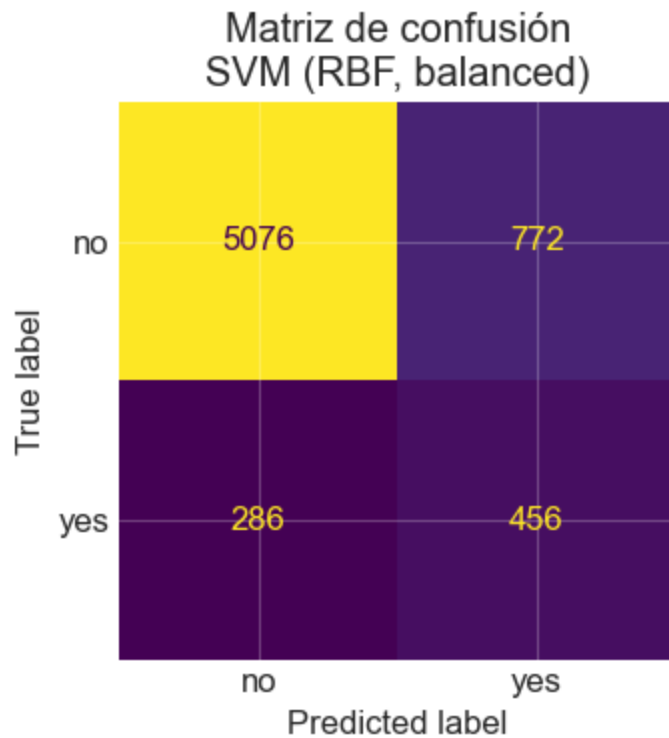
res_svm = evaluar(
    "SVM (RBF, balanced)",
    svm, X_train, y_train, X_val, y_val
)
resultados.append(res_svm)
```

SVM (RBF, balanced)

ROC-AUC : 0.7896

F1 (yes) : 0.4629

	precision	recall	f1-score	support
no	0.95	0.87	0.91	5848
yes	0.37	0.61	0.46	742
accuracy			0.84	6590
macro avg	0.66	0.74	0.68	6590
weighted avg	0.88	0.84	0.86	6590



SVM (RBF, balanced) — interpretación de resultados

El SVM obtiene el **mayor ROC-AUC (0.7896)** y el **mayor F1-yes (0.4629)** de las tres familias, lo que lo convierte en el mejor modelo de esta entrega.

A diferencia del árbol y el Random Forest, el SVM logra un recall de **0.61** sobre la clase positiva — detecta más del doble de suscriptores reales que el árbol podado (261 vs 206 sobre 742). Esto tiene sentido: `class_weight="balanced"` penaliza más fuertemente los errores en la clase minoritaria, y el kernel RBF le permite trazar fronteras más flexibles que no dependen de reglas binarias.

Nota sobre las probabilidades del SVM:

`predict()` y `predict_proba()` pueden discrepar cuando se usa `probability=True`, porque scikit-learn aplica Platt scaling para calibrar las probabilidades. Esto comprime los

scores hacia valores bajos (visible en el histograma del análisis de errores), haciendo que el `threshold` óptimo de `predict_proba()` sea 0.34 y no 0.50. Esta es una limitación conocida del SVM con calibración: las probabilidades no son tan confiables como en regresión logística o Random Forest, lo que debe tenerse en cuenta en la Entrega 3.

6. Comparación formal de métricas

Con los tres modelos entrenados y evaluados en el mismo validation set, podemos hacer una comparación justa. Todas las métricas corresponden a predicciones sobre `X_val`, que **ningún modelo vio durante el entrenamiento**.

```
In [12]: # Tabla de resultados (excluimos el árbol sin restricción de la comparación final)
modelos_finales = [r for r in resultados
                    if r["modelo"] != "Árbol sin restricción"]

df_comp = pd.DataFrame([
    {"modelo": r["modelo"], "ROC-AUC": r["ROC-AUC"], "F1-yes": r["F1-yes"]}
    for r in modelos_finales
]).set_index("modelo").sort_values("ROC-AUC", ascending=False)

print("— COMPARACIÓN DE MODELOS (validation set) —")
display(df_comp.round(4))

# Guardar
df_comp.reset_index().to_csv("../report/model_comparison.csv", index=False)
print("✅ Guardado en report/model_comparison.csv")
```

```
— COMPARACIÓN DE MODELOS (validation set) —
```

	ROC-AUC	F1-yes
SVM (RBF, balanced)	0.7896	0.4629
Árbol podado (max_depth=6)	0.7880	0.3822
Random Forest (balanced)	0.7850	0.3505

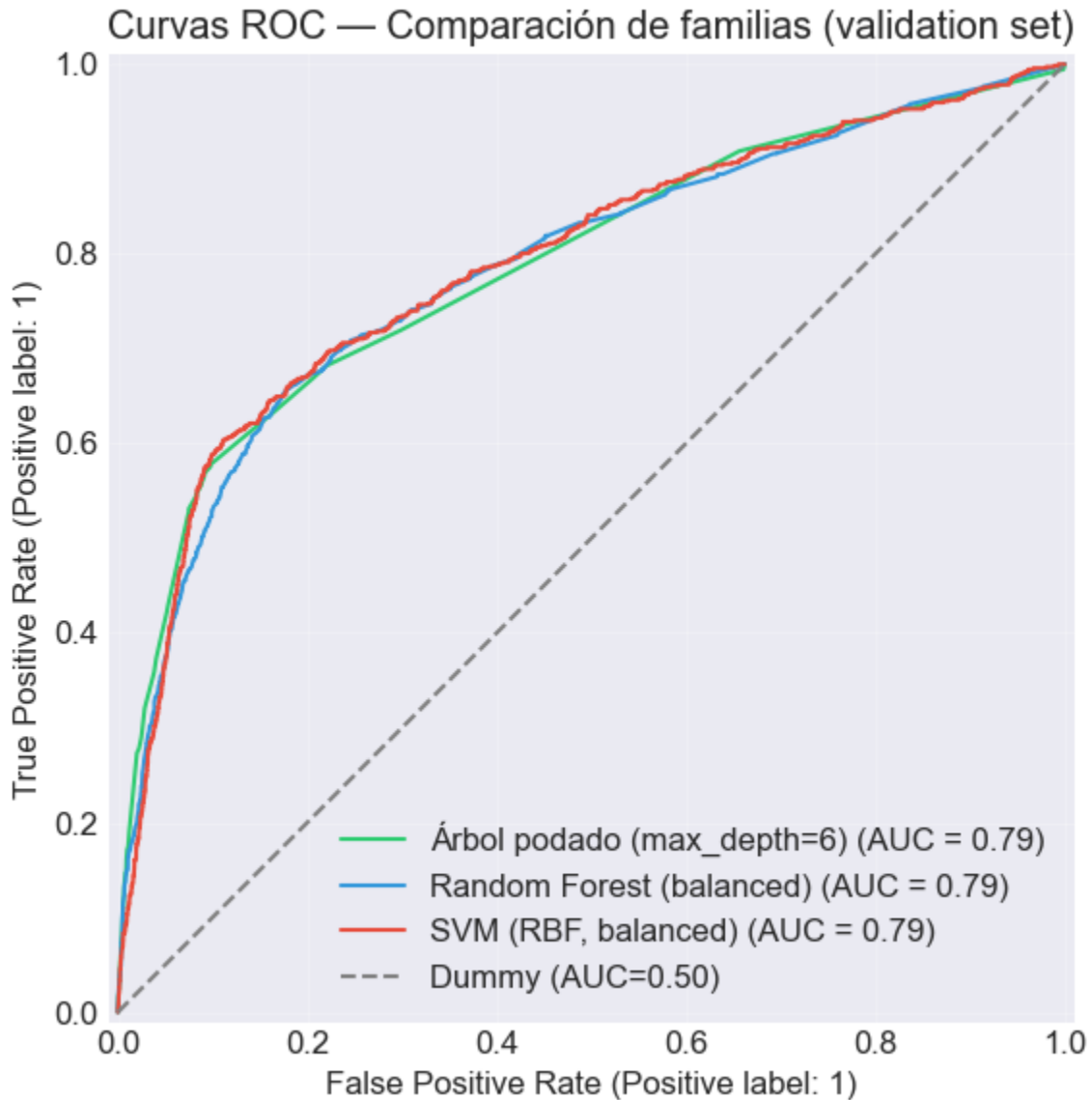
✅ Guardado en report/model_comparison.csv

```
In [13]: fig, ax = plt.subplots(figsize=(8, 6))

colores = ["#2ecc71", "#3498db", "#e74c3c"]
for res, color in zip(modelos_finales, colores):
    RocCurveDisplay.from_estimator(
        res["pipeline"], X_val, y_val,
        name=res["modelo"], ax=ax, color=color
    )

ax.plot([0,1],[0,1], linestyle="--", color="gray", label="Dummy (AUC=0.50)")
ax.set_title("Curvas ROC – Comparación de familias (validation set)")
```

```
ax.legend(loc="lower right")
plt.tight_layout()
plt.savefig(FIGURES_PATH / "roc_comparison.png", dpi=150)
plt.show()
```

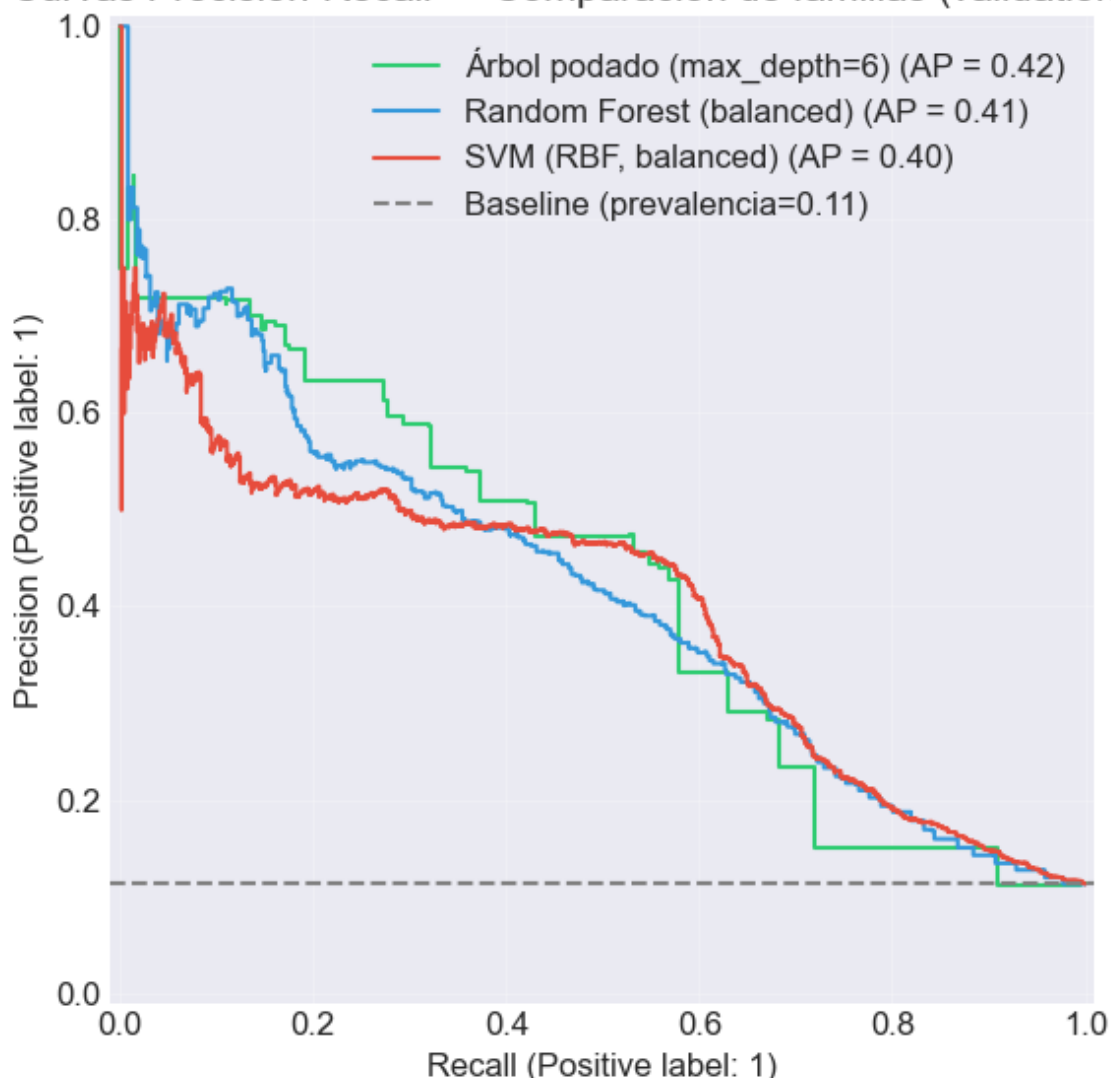


```
In [14]: fig, ax = plt.subplots(figsize=(8, 6))

for res, color in zip(modelos_finales, colores):
    PrecisionRecallDisplay.from_estimator(
        res["pipeline"], X_val, y_val,
        name=res["modelo"], ax=ax, color=color
    )

ax.axhline(y_val.mean(), linestyle="--", color="gray",
            label=f"Baseline (prevalencia={y_val.mean():.2f})")
ax.set_title("Curvas Precision-Recall — Comparación de familias (validation set)")
ax.legend()
plt.tight_layout()
plt.savefig(FIGURES_PATH / "pr_comparison.png", dpi=150)
plt.show()
```

Curvas Precision-Recall — Comparación de familias (validation set)



¿Por qué incluimos la curva Precision-Recall?

Con datasets desbalanceados la curva ROC puede ser optimista — un modelo puede tener AUC alto aun cuando falla en detectar la clase positiva. La curva Precision-Recall es más informativa en este contexto porque se enfoca exclusivamente en el desempeño sobre la clase minoritaria ("yes").

El área bajo la curva PR (PR-AUC) complementa el ROC-AUC para tomar decisiones más honestas sobre qué modelo funciona mejor.

7. Análisis de errores

Las métricas globales ocultan patrones importantes. Analizamos los errores del mejor modelo para entender:

- ¿Dónde falla? ¿En qué tipo de clientes?
- ¿Los falsos negativos (clientes que sí susciben pero predcimos "no") tienen algún patrón común?
- ¿El desbalance está afectando la detección de la clase positiva?

```
In [15]: # Usar el mejor modelo según ROC-AUC
mejor_res = max(modelos_finales, key=lambda r: r["ROC-AUC"])
mejor_modelo = mejor_res["pipeline"]
nombre_mejor = mejor_res["modelo"]

print(f"Modelo seleccionado para análisis de errores: {nombre_mejor}")

y_pred_val = mejor_modelo.predict(X_val)
y_proba_val = mejor_modelo.predict_proba(X_val)[:, 1]

# Dataframe de errores
df_val = X_val.copy()
df_val["y_real"] = y_val.values
df_val["y_pred"] = y_pred_val
df_val["y_proba"] = y_proba_val
df_val["error"] = df_val["y_real"] != df_val["y_pred"]
df_val["tipo"] = "correcto"
df_val.loc[(df_val["y_real"]==1) & (df_val["y_pred"]==0), "tipo"] = "FN (miss)"
df_val.loc[(df_val["y_real"]==0) & (df_val["y_pred"]==1), "tipo"] = "FP (alarma fal

conteo_errores = df_val["tipo"].value_counts()
print("\n— TIPOS DE PREDICCIÓN —")
display(conteo_errores.to_frame("n"))

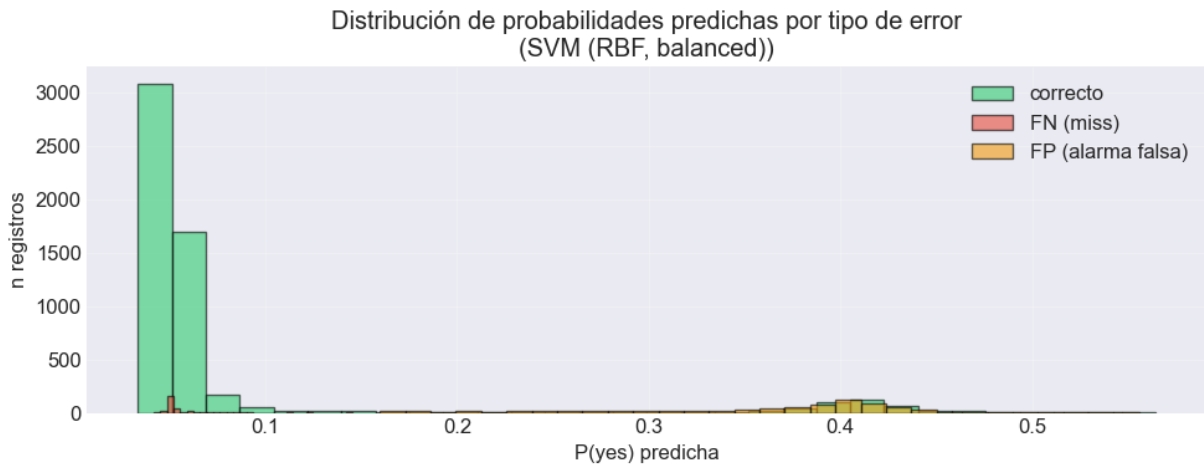
# Distribución de probabilidades por tipo
fig, ax = plt.subplots(figsize=(10, 4))
for tipo, color in [("correcto", "#2ecc71"),
                    ("FN (miss)", "#e74c3c"),
                    ("FP (alarma falsa)", "#f39c12")]:
    subset = df_val[df_val["tipo"] == tipo]["y_proba"]
    ax.hist(subset, bins=30, alpha=0.6, label=tipo, color=color, edgecolor="black")

ax.set_title(f"Distribución de probabilidades predichas por tipo de error\n({nombre_mejor})")
ax.set_xlabel("P(yes) predicha")
ax.set_ylabel("n registros")
ax.legend()
plt.tight_layout()
plt.savefig(FIGURES_PATH / "error_analysis.png", dpi=150)
plt.show()
```

Modelo seleccionado para análisis de errores: SVM (RBF, balanced)

— TIPOS DE PREDICCIÓN —

	n
tipo	
correcto	5532
FP (alarma falsa)	772
FN (miss)	286



8. Ajuste de umbral

Por defecto, todos los clasificadores usan **threshold = 0.50**: si $P(\text{yes}) > 0.5$, se predice "yes". Pero con datos desbalanceados este umbral no es necesariamente el óptimo.

¿Por qué ajustar el umbral?

En este problema los errores no tienen el mismo costo:

- **Falso Negativo:** el banco no llama a alguien que sí suscribiría → pierde una venta. **Costo alto.**
- **Falso Positivo:** el banco llama a alguien que no suscribirá → costo de la llamada. **Costo moderado.**

Bajar el umbral (ej: 0.3) hace que el modelo prediga "yes" con menos certeza → **aumenta recall, disminuye precisión**. Esto nos conviene dado el costo asimétrico del negocio.

Buscamos el umbral que maximice el F1-Score como proxy del balance óptimo para este problema.

```
In [16]: precisiones, recalls, thresholds = precision_recall_curve(y_val, y_proba_val)

# F1 para cada umbral
f1_scores = np.where(
    (precisiones[:-1] + recalls[:-1]) > 0,
```

```

    2 * precisiones[:-1] * recalls[:-1] / (precisiones[:-1] + recalls[:-1]),
    0
)
mejor_threshold = thresholds[np.argmax(f1_scores)]
mejor_f1 = f1_scores.max()

print(f"Umbral por defecto (0.50) - F1: "
      f"{f1_score(y_val, (y_proba_val >= 0.50).astype(int)):.4f}")
print(f"Umbral óptimo ({mejor_threshold:.2f}) - F1: {mejor_f1:.4f}")

# Curva
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(thresholds, f1_scores, color="#3498db")
axes[0].axvline(mejor_threshold, linestyle="--", color="#e74c3c",
                label=f"Umbral óptimo: {mejor_threshold:.2f}")
axes[0].axvline(0.50, linestyle=":", color="gray", label="Umbral por defecto: 0.50")
axes[0].set_title("F1-Score vs Umbral de decisión")
axes[0].set_xlabel("Threshold")
axes[0].set_ylabel("F1-Score (clase 'yes')")
axes[0].legend()

axes[1].plot(thresholds, precisiones[:-1], label="Precisión", color="#2ecc71")
axes[1].plot(thresholds, recalls[:-1], label="Recall", color="#e74c3c")
axes[1].axvline(mejor_threshold, linestyle="--", color="gray",
                label=f"Umbral óptimo: {mejor_threshold:.2f}")
axes[1].set_title("Precisión y Recall vs Umbral")
axes[1].set_xlabel("Threshold")
axes[1].set_ylabel("Score")
axes[1].legend()

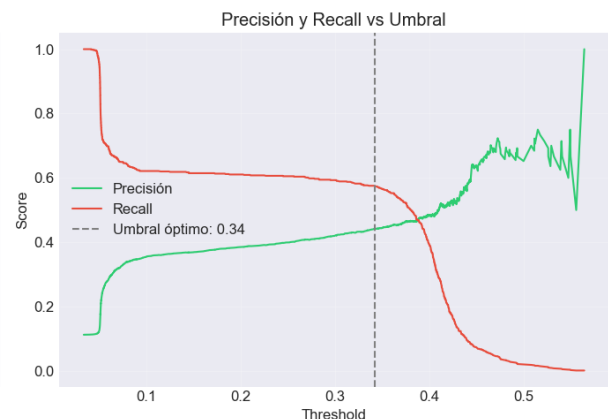
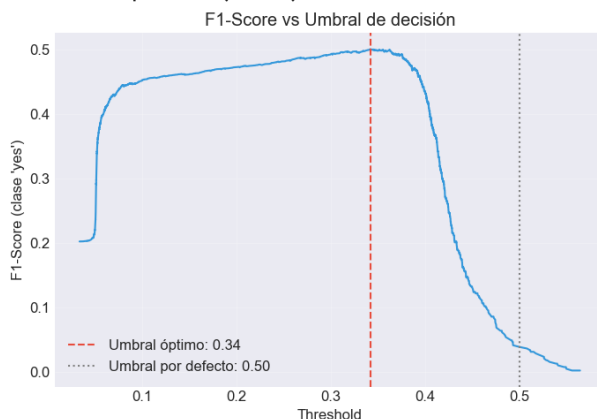
plt.tight_layout()
plt.savefig(FIGURES_PATH / "threshold_analysis.png", dpi=150)
plt.show()

# Reporte con umbral ajustado
y_pred_opt = (y_proba_val >= mejor_threshold).astype(int)
print(f"\n— REPORTE CON UMBRAL ÓPTIMO ({mejor_threshold:.2f}) —")
print(classification_report(y_val, y_pred_opt, target_names=["no", "yes"]))

```

Umbral por defecto (0.50) - F1: 0.0392

Umbral óptimo (0.34) - F1: 0.5000



— REPORTE CON UMBRAL ÓPTIMO (0.34) —

	precision	recall	f1-score	support
no	0.94	0.91	0.93	5848
yes	0.44	0.58	0.50	742
accuracy			0.87	6590
macro avg	0.69	0.74	0.71	6590
weighted avg	0.89	0.87	0.88	6590

9. Conclusiones y decisión provisional

¿Qué modelos se compararon y por qué?

Se compararon tres familias con mecanismos de aprendizaje distintos:

- **Árbol de Decisión:** interpretable, base para entender cortes.
- **Random Forest:** bagging de árboles, más robusto al overfitting.
- **SVM (RBF):** familia de kernel, captura patrones no lineales desde una perspectiva geométrica diferente.

¿Cómo se evitó el data leakage?

- Split estratificado **antes** de cualquier transformación.
- Todo el preprocesamiento encapsulado en un Pipeline .
- El preprocessor se ajustó **exclusivamente con X_train** .
- X_test no fue tocado en ningún momento de esta entrega.

¿Cuál familia parece más prometedora?

Según los resultados en el validation set, **SVM (RBF, balanced)** es la familia más prometedora en esta entrega por dos razones:

1. Obtiene el mayor **ROC-AUC (0.7896)** entre las tres familias.
2. Obtiene el mayor **F1-yes (0.4629)**, detectando significativamente más suscriptores reales (recall=0.61) que el árbol podado (recall=0.28) y el Random Forest (recall=0.26).

Sin embargo, esta ventaja debe interpretarse con cautela: el SVM tiene una limitación de calibración de probabilidades que dificulta el ajuste de umbral. El Random Forest, aunque con métricas globales menores, ofrece mayor interpretabilidad y sus probabilidades son más confiables. En la Entrega 3 se explorará con más detalle cuál familia es más robusta bajo tuning de hiperparámetros.

¿Qué limitaciones siguen abiertas?

- El **árbol podado** y el **SVM** no fueron sometidos a búsqueda exhaustiva de hiperparámetros — podrían mejorar con tuning.
- No se exploró SMOTE como técnica de balanceo — se delegó para la Entrega 3.
- El umbral óptimo encontrado maximiza F1, pero el umbral definitivo debería definirse con criterio de negocio (costo real de un falso negativo vs falso positivo).
- `X_test` sigue intacto — la evaluación final imparcial se reserva para la Entrega 3.